

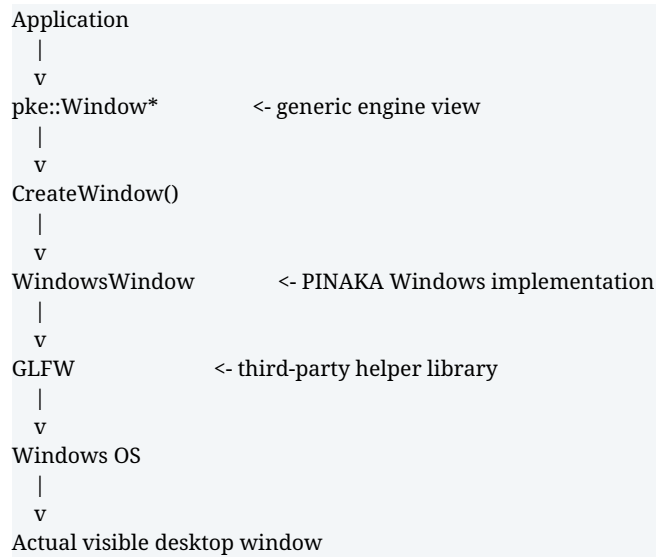
PINAKA

WINDOW v1

From a generic Window interface to PINAKA's first real GLFW window

Goal: understand the Window subsystem intuitively - what problem it solves, how Application uses it, where GLFW fits, and how the real desktop window is created, updated, closed, and destroyed.

The core mental model



1. Why PINAKA Needs a Window Subsystem

Application v1 gave PINAKA a lifecycle: initialize, run a loop, update, and shut down. But there was still no real desktop window. A game engine or editor needs a place where the user can see and interact with the program.

The Window subsystem is the bridge between the high-level engine and the operating system.

Simple meaning: Window is the engine's doorway to the desktop environment.

Application says:

"I need a window."

Window system says:

"I will give you a generic Window."

Platform code says:

"On Windows, I know how to create the real one."

GLFW says:

"I will handle the low-level operating-system work for you."

2. Folder Structure and Responsibility

```
PinakaEngine/
├── src/
│   ├── Pinaka/
│   │   ├── Foundation/
│   │   ├── Core/
│   │   │   ├── Application/
│   │   │   │   ├── Application.h
│   │   │   │   ├── Application.cpp
│   │   │   │   └── EntryPoint.h
│   │   └── Window/
│   │       └── Window.h
│   └── Platform/
│       ├── Windows/
│       │   ├── WindowsWindow.h
│       │   └── WindowsWindow.cpp
```

Core contains the generic engine idea. Platform contains the operating-system-specific implementation.

Location	Meaning
Core/Window/Window.h	What a Window means to PINAKA
Platform/Windows/WindowsWindow.*	How that Window is implemented on Windows

Core should not contain Windows-specific code. Higher-level engine code should depend on the generic Window interface.

3. Why Window Is an Interface

PINAKA should not force Application to know whether it is running on Windows, Linux, or macOS. Instead, Application talks to one common type: `pke::Window`.

```
pke::Window
^
|
```

```

+-- WindowsWindow
+-- LinuxWindow  (future)
+-- MacWindow    (future)

```

This means Application can use the same functions no matter which concrete platform implementation exists underneath.

4. Window.h - The v1 Interface

```

namespace pke
{
    class PK_API Window
    {
    public:
        virtual unsigned int width() const = 0;
        virtual unsigned int height() const = 0;

        virtual bool isOpen() const = 0;

        virtual void update() = 0;

        virtual ~Window() = default;
    };

    Window* CreateWindow();
}

```

This interface is intentionally small. It contains only the minimum behavior Application currently needs.

What each function means

Function	Simple meaning	Type of operation
width()	How wide is the window?	Query
height()	How tall is the window?	Query
isOpen()	Is this window still allowed to remain open?	Query
update()	Process the window's work for this frame	Per-frame action

5. Why We Removed Mouse, Resize, and Close Callbacks from Window.h

The first Window design mixed two different ideas: commands/queries and events.

Commands / queries:

- "What is your width?"
- "Are you open?"
- "Update yourself."

Events:

- "The user clicked X."
- "The window was resized."
- "The mouse was double-clicked."
- "A key was pressed."

Things that happen to the window are better represented by the Event system, which we will build next.

Window describes what the engine can do with or ask from a window. Events describe what happened.

6. Why width() and height() Use unsigned int

Window dimensions are normally whole pixel counts such as 1280 x 720 or 1920 x 1080. A floating-point type such as double is unnecessary for the current design.

```
1280 pixels
720 pixels

not:

1280.417 pixels
```

That is why unsigned int is a reasonable v1 choice for width and height.

7. Why isOpen() Is const

```
virtual bool isOpen() const = 0;
```

isOpen() only asks a question. It should not modify the object. The const keyword expresses that promise.

Const intuition: 'I am only looking at the state, not changing it.'

8. WindowsWindow - The Concrete Platform Class

pke::Window is abstract, so it cannot be instantiated directly. WindowsWindow provides the actual Windows-side implementation.

```
class WindowsWindow : public Window
{
public:
    WindowsWindow();
    WindowsWindow(int width, int height, const char* name);

    unsigned int width() const override;
    unsigned int height() const override;
    bool isOpen() const override;
    void update() override;

    ~WindowsWindow() override;

private:
    unsigned int m_width;
    unsigned int m_height;
    const char* m_name;

    GLFWwindow* m_pWindow;
};
```

The override keyword is important because it asks the compiler to verify that each function really overrides a matching virtual function from Window.

9. CreateWindow() - Hiding the Concrete Platform Type

Application should not do this:

```
WindowsWindow* window = new WindowsWindow(); // too platform-specific for Application
```

Instead, the generic Window API declares a creation function:

```
Window* CreateWindow();
```

and the Windows implementation defines it:

```
Window* CreateWindow()
{
    return new WindowsWindow();
}
```

Application receives only a Window pointer.

```
Window* m_pWindow;

m_pWindow = CreateWindow();
```

Application asks for a Window, not specifically a WindowsWindow. Platform code decides what concrete object to create.

10. Why Application Stores Window* Instead of WindowsWindow*

```
Window* m_pWindow;
```

This is the central abstraction boundary. Application only needs the Window interface.

Application can call:

```
m_pWindow->width();
m_pWindow->height();
m_pWindow->isOpen();
m_pWindow->update();
```

Application does NOT need to know:

```
WindowsWindow
GLFWwindow
Win32 APIs
```

The actual object underneath the pointer can still be WindowsWindow because derived objects can be referenced through a base-class pointer.

The type Application sees is Window*. The real object is WindowsWindow.

11. Application Owns the Window

Application creates the Window during initialization, uses it while running, and deletes it during shutdown.

```
void Application::initEngine()
{
    m_pWindow = CreateWindow();
    m_Running = true;
}

void Application::shutdown()
{
    delete m_pWindow;
    m_pWindow = nullptr;
}
```

This means Application currently has single ownership of the Window resource.

Later, UniquePtr<Window> is a natural improvement because Application has one clear owner. For v1, the raw pointer makes the ownership flow easy to see.

12. Connecting Window State to the Application Loop

```
void Application::run()
{
    initEngine();

    while (m_Running && m_pWindow->isOpen())
    {
        m_pWindow->update();
        update();
    }

    shutdown();
}
```

The loop now has two reasons to stop:

- Application itself requested shutdown, so m_Running becomes false.
- The Window is no longer open, so isOpen() becomes false.

```
while (Application wants to run AND Window is open)
{
    update the Window
    update the client Application
}
```

This is the first real connection between Application lifecycle and Window lifecycle.

13. GLFW From Absolute Zero

GLFW is a third-party library that helps PINAKA create desktop windows and process operating-system input/window events without writing large amounts of Win32 code directly.

```
PINAKA
|
v
WindowsWindow
|
v
GLFW
|
v
Windows OS
```

Without GLFW, creating a Windows window directly would involve Win32 concepts such as HWND, WNDCLASS, CreateWindowEx, WndProc, and message processing. Those are valuable topics, but they are not required for Window v1.

GLFW is not replacing our Window abstraction. GLFW sits underneath WindowsWindow as an implementation helper.

14. GLFW Step 1 - glfwInit()

```
glfwInit();
```

glfwInit() prepares the GLFW library for use. It initializes GLFW itself, not one individual window.

```
if (!glfwInit())
{
    // GLFW could not initialize
}
```

It can fail, so the return value should be checked.

Mental model: glfwInit() = 'GLFW, get yourself ready because PINAKA is about to use you.'

15. GLFW Step 2 - GLFWwindow*

GLFW defines a type named GLFWwindow. PINAKA does not construct it directly. GLFW creates the object and returns a pointer.

```
GLFWwindow* m_pWindow;
```

This member represents the actual GLFW-managed desktop window resource.

```
WindowsWindow object
|
| owns/wraps
v
GLFWwindow*
|
v
real desktop window
```

WindowsWindow is our engine object. GLFWwindow* is GLFW's handle/pointer to the real window resource.

16. Keeping GLFW Out of Generic Headers

Generic Window.h should not include GLFW. Studio should not need a GLFW include path merely because it uses pke::Window.

```
Core/Window/Window.h
-> no GLFW include

Platform/Windows/WindowsWindow.h
-> may forward-declare GLFWwindow

Platform/Windows/WindowsWindow.cpp
-> includes <GLFW/glfw3.h>
```

A forward declaration is enough in WindowsWindow.h if the header only stores a pointer:

```
struct GLFWwindow;

class WindowsWindow : public Window
{
    ...
    GLFWwindow* m_pWindow;
};
```

Then the full GLFW header is included in the .cpp file where GLFW functions are actually called.

Implementation detail rule: do not leak third-party dependencies upward unless higher-level code truly needs them.

17. GLFW Step 3 - glfwCreateWindow()

```
m_pWindow = glfwCreateWindow(
    m_width,
    m_height,
    m_name,
    nullptr,
    nullptr
);
```

Read the call in plain English:

```
GLFW:
"Create a window that is:
  width = m_width
  height = m_height
  title = m_name

and give me back a GLFWwindow*."
```

What the five arguments mean

Argument	Simple meaning
m_width	Window width
m_height	Window height
m_name	Text shown in the title bar
nullptr (monitor)	Normal windowed mode, not full-screen
nullptr (share)	Do not share an OpenGL context with another window yet

glfwCreateWindow() can fail even after glfwInit() succeeds. If creation fails, it returns nullptr.

```
if (!m_pWindow)
{
    // Window creation failed
}
```

18. Constructor Flow

```
WindowsWindow::WindowsWindow()
: m_width(1280),
  m_height(720),
  m_name("Pinaka"),
  m_pWindow(nullptr)
{
    if (!glfwInit())
        return;

    m_pWindow = glfwCreateWindow(
        m_width,
        m_height,
        m_name,
        nullptr,
        nullptr
    );
}
```

m_pWindow must start as nullptr. Otherwise, if glfwInit() fails before glfwCreateWindow() is called, the pointer would contain an unknown garbage value.

Always give owned pointer members a known initial state, usually nullptr.

19. GLFW Step 4 - glfwPollEvents()

```
void WindowsWindow::update()
{
    glfwPollEvents();
}
```

glfwPollEvents() asks GLFW to process operating-system events that happened since the previous frame.

- User clicked the close button.
- User moved or resized the window.
- Keyboard input happened.
- Mouse input happened.

For Window v1, we only use the close state. The richer events will be handled by the Event system later.

Mental model: glfwPollEvents() = 'GLFW, check what happened on the desktop since the last frame.'

20. GLFW Step 5 - glfwWindowShouldClose()

```
bool WindowsWindow::isOpen() const
{
    return !glfwWindowShouldClose(m_pWindow);
}
```

GLFW tracks whether the window has been asked to close. glfwWindowShouldClose() returns true when closing has been requested.

```
glfwWindowShouldClose(...) == true
means:
```

```
"this window should close"

therefore:

!glfwWindowShouldClose(...)
means:
"this window is still open"
```

Once `isOpen()` uses GLFW directly, the old separate `m_isOpen` boolean is no longer necessary.

21. Closing the Real Window - Destructor and RAI

The GLFW window must not be destroyed with C++ `delete` because GLFW created it internally.

```
delete m_pWindow; // WRONG for GLFWwindow*
```

Use GLFW's matching destruction function:

```
WindowsWindow::~~WindowsWindow()
{
    if (m_pWindow)
    {
        glfwDestroyWindow(m_pWindow);
        m_pWindow = nullptr;
    }

    glfwTerminate();
}
```

For the current one-window v1 implementation, calling `glfwTerminate()` here is acceptable as a simple learning design.

Matching resource pair: `glfwCreateWindow()` -> `glfwDestroyWindow()`. This is RAI in a real engine context.

Later, if PINAKA supports multiple GLFW windows, initialization and termination should be moved to a shared GLFW/platform lifetime manager. Destroying one window must not terminate the whole GLFW library while other windows still exist.

22. The Complete Window Runtime Flow

```
PinakaStudio.exe starts
|
v
EntryPoint main()
|
v
CreateApplication()
|
v
StudioApplication
|
v
Application::run()
|
v
Application::initEngine()
|
v
CreateWindow()
|
v
```

```

new WindowsWindow()
|
v
glfwInit()
|
v
glfwCreateWindow(...)
|
v
VISIBLE WINDOW APPEARS
|
v
while (m_Running && m_pWindow->isOpen())
|
+--> m_pWindow->update()
|   |
|   v
|   glfwPollEvents()
|
+--> StudioApplication::update()
|
v
User clicks X
|
v
GLFW marks window 'should close'
|
v
isOpen() returns false
|
v
Application loop ends
|
v
Application::shutdown()
|
v
delete Window*
|
v
WindowsWindow destructor
|
+--> glfwDestroyWindow()
+--> glfwTerminate()
|
v
Application returns to main()
|
v
PinakaStudio.exe exits

```

23. GLFW Build Integration in Premake

GLFW was added as a Git submodule at:

```
D:\Aman\Pinaka\thirdparty\GLFW\glfw
```

PINAKA builds GLFW as its own static library project and links it into PinakaEngine.

```
GLFW      -> StaticLib
PinakaEngine  -> SharedLib / DLL
PinakaStudio  -> ConsoleApp / EXE
Tests       -> ConsoleApp / EXE

GLFW.lib
|
v
PinakaEngine.dll
|
v
PinakaStudio.exe
```

Two Premake concepts worth remembering

Premake setting	Meaning
includedirs	Compiler: where can I find the header files?
links	Linker: which compiled library contains the function implementations?

```
includedirs
-> .h files
-> compilation

links
-> .lib
-> linking
```

24. Why Studio Does Not Link GLFW Directly

Studio uses PinakaEngine. PinakaEngine uses GLFW. Studio should not need to know that Window happens to be implemented using GLFW.

```
Studio
|
v
PinakaEngine
|
v
pke::Window
|
v
WindowsWindow
|
v
GLFW
```

Abstraction boundary: Studio depends on PINAKA, not directly on PINAKA's third-party implementation details.

25. Why GLFW Needed Win32 and Null Backend Source Files

Current GLFW versions compile common platform-selection code that can reference both the selected Win32 backend and GLFW's null backend. When the null backend source files were missing from the custom Premake project, the linker reported an unresolved symbol such as `_glfwConnectNull`.

Required common/null files included in the GLFW project:

```
null_init.c
null_monitor.c
null_window.c
null_joystick.c
```

The Windows build also defines `_GLFW_WIN32` and includes the Win32 source files such as `win32_window.c` and `wgl_context.c`.

26. Problems We Hit and What They Taught Us

Problem A - Application did not know Window

`Application.h` stored `Window*` but the compiler did not yet know `Window` was a class.

```
namespace pke
{
    class Window; // forward declaration
}
```

Because `Application.h` stores only a pointer, a forward declaration is enough there. `Application.cpp` includes the full `Window.h` because it calls `Window` functions and deletes the object.

Problem B - GLFW leaked into generic Window.h

Including `<GLFW/glfw3.h>` in generic `Window.h` caused Studio to require GLFW headers too. Moving the GLFW include down into `WindowsWindow.cpp` restored the abstraction boundary.

Problem C - `_glfwConnectNull` linker error

The custom GLFW static-library build was missing the null backend source files. Adding them fixed the unresolved external symbol.

Problem D - Premake expected a closing brace

A missing comma after `osmesa_context.c` made Lua think the files table had invalid syntax. This was a small build-script syntax issue, not a GLFW problem.

Problem E - uninitialized `GLFWwindow*` warning

If `glfwInit()` failed before `m_pWindow` was assigned, the pointer could remain uninitialized. Initializing it to `nullptr` in the constructor fixed the ownership state.

27. What Window v1 Has Achieved

- A clean generic `pke::Window` interface.
- A Windows-specific concrete implementation.
- A factory-style `CreateWindow()` function that hides the concrete type.
- Application owns and updates a generic `Window` pointer.
- GLFW is integrated as a third-party static library.
- GLFW initializes successfully.

- PINAKA creates a real visible desktop window.
- The window processes OS events every frame.
- Clicking X makes isOpen() become false.
- The Application loop exits cleanly.
- The GLFW window is destroyed during shutdown.

Window v1 is complete enough to move forward. It is a working subsystem, not just an interface sketch.

28. What Window v1 Does NOT Do Yet

- No PINAKA Event objects yet.
- No WindowCloseEvent or WindowResizeEvent yet.
- No keyboard or mouse event classes yet.
- No renderer or graphics API setup yet.
- No OpenGL/Vulkan rendering context decisions yet.
- No multiple-window lifetime management yet.
- No proper shared GLFW initialization manager yet.
- No configurable WindowProps structure yet.
- No smart-pointer ownership for Application's Window yet.

These are future improvements, not missing requirements for v1.

29. The Next Natural Step - Events

Right now Application checks Window state directly:

```
while (m_Running && m_pWindow->isOpen())
```

The next architecture layer will allow Window to communicate what happened:

```
User clicks X
|
v
GLFW receives it
|
v
WindowsWindow
|
v
WindowCloseEvent
|
v
Application
|
v
requestShutdown()
```

Window v1 answered: 'Can PINAKA create and maintain a real window?' Events v1 will answer: 'How do engine systems communicate what happened?'

30. Interview Quick Revision - Window v1

30-second explanation

PINAKA exposes a generic `pke::Window` interface inside Core and implements it with `WindowsWindow` in the Windows platform layer. Application stores only a `Window*` and obtains the concrete implementation through `CreateWindow()`, so it is not coupled to Windows-specific code. `WindowsWindow` wraps a `GLFWwindow*`, uses GLFW to create the real desktop window, polls OS events every frame, checks the GLFW close flag through `isOpen()`, and destroys the GLFW resource during shutdown.

Key concepts

- Abstraction: Application sees `Window`, not `WindowsWindow`.
- Inheritance and polymorphism: `WindowsWindow` implements the `Window` interface.
- Factory-style creation: `CreateWindow()` hides the concrete platform class.
- Encapsulation: GLFW details stay in the platform implementation.
- Forward declaration: `Window` and `GLFWwindow` pointer types can be declared without full headers in some headers.
- RAI: `WindowsWindow` lifetime owns the GLFW window resource.
- Dependency direction: Studio -> Engine -> Window -> `WindowsWindow` -> GLFW.
- Build separation: GLFW is a static library linked by `PinakaEngine`.

Likely interview questions

Why not store `WindowsWindow*` in Application? Because Application should depend on the generic `Window` abstraction, not one operating system.

Why is `Window` abstract? Different platforms can provide different implementations while higher-level engine code uses one common API.

Why `CreateWindow()`? It hides platform-specific object construction from Application.

Why does `WindowsWindow` store `GLFWwindow*`? GLFW creates and manages the actual native window resource; `WindowsWindow` wraps it inside PINAKA.

What does `glfwInit()` do? Initializes the GLFW library as a whole.

What does `glfwCreateWindow()` return? A `GLFWwindow*` representing the created window, or `nullptr` on failure.

What does `glfwPollEvents()` do? Processes pending OS/window/input events for the current frame.

What does `glfwWindowShouldClose()` do? Reports whether the window has been requested to close.

Why not delete `GLFWwindow*`? GLFW created it, so it must be destroyed with `glfwDestroyWindow()`.

Why not include GLFW in `Window.h`? GLFW is a lower-level implementation detail; leaking it upward couples generic engine/client code to the third-party library.

Memory hook

Application asks for a `Window`. Platform code gives it a `WindowsWindow`. `WindowsWindow` uses GLFW. GLFW talks to Windows.

Final flow to remember

Application
-> `CreateWindow()`

- > WindowsWindow
- > glfwInit()
- > glfwCreateWindow()
- > visible window

Every frame:

- > glfwPollEvents()
- > isOpen()

On X:

- > GLFW says should close
- > isOpen() becomes false
- > Application loop ends
- > delete Window
- > glfwDestroyWindow()
- > glfwTerminate()

WINDOW v1 - COMPLETE